

CONTROL DE VERSIONES Y TRABAJO EN EQUIPO

Git & GitHub para Proyectos Colaborativos

Flujos de trabajo con Forks, integración mediante Pull Requests y gestión de estándares con CONTRIBUTING.md.

 Ing. Sergio Andrés Noh Puch  Metodologías Open Source

Sistemas de Control de Versiones (VCS)



Modelo Centralizado (SVN)

Un único servidor alberga el historial completo del proyecto.

-
- ✗ Único punto de falla (Single point of failure)
 - ✗ Operaciones lentas y dependencia de red



Modelo Distribuido (Git)

Cada colaborador posee una copia local completa de la base de datos.

-
- ✓ Autonomía total y trabajo fuera de línea
 - ✓ Respaldo redundante e historial local

Git como Motor, GitHub como Plataforma

Git

Motor local de control de versiones
distribuido (2005)

GitHub

Plataforma en la nube para alojar y
colaborar

100M+

Desarrolladores e investigadores
conectados

GitHub proporciona la infraestructura social para revisión de código, seguimiento de issues y automatización de flujos de trabajo (CI/CD).

Anatomía de un Repositorio ()

Carpeta oculta **.git**: Base de datos del proyecto

Historial de Cambios: Almacena commits, ramas (branches) y etiquetas (tags).

Archivos Planos vs. Binarios: Git optimiza diferencias línea por línea en texto plano (.py, .cpp, .md).

Desafío Binario: Archivos CAD (.STL, .STEP), PDFs o esquemáticos (.PCB) requieren almacenamiento especializado (Git LFS).

Buenas Prácticas en Repositorios

 **.gitignore:** Excluye archivos temporales y ejecutables generados.

 **README.md:** Carta de presentación y guía de uso del proyecto.

 **CONTRIBUTING.md:** Normas de participación colaborativa.

El Modelo de Trabajos con Forks



¿Qué es un Fork?

Una copia personal e independiente de un repositorio ajeno alojada en tu cuenta de GitHub.

Permite experimentar y modificar código libremente sin alterar el repositorio base ni requerir permisos directos de escritura.

Ventajas del Trabajo con Forks

Entorno de Sandbox Seguro

Cero riesgo de romper la rama principal del proyecto original.

Control de Acceso Simplificado

Los mantenedores no necesitan gestionar permisos de colaboradores externos.

Canalización Vía Pull Request

Facilita la revisión, discusión y auditoría de aportes externos.

Flujo Colaborativo: Fork & Pull Request

PASO 01

Fork del Repositorio

Clonas el proyecto original hacia tu propia cuenta remota de GitHub.

PASO 02

Rama de Trabajo

Creas una rama local descriptiva (feature/nueva-mejora) para editar.

PASO 03

Push a tu Fork

Subes los cambios realizados localmente a tu copia en la nube.

PASO 04

Pull Request (PR)

Solicitas al mantenedor integrar tus cambios tras la revisión de código.

Gestión de Colaboración: CONTRIBUTING.md

Estándares de Código

Define el formato, linters y convenciones de estilo de programación obligatorias.

Formato de Commits

Establece reglas para mensajes claros (ej. Conventional Commits: feat:, fix:).

Pruebas y Verificación

Exige pasar tests unitarios y validaciones automáticas antes de enviar el PR.

Plantillas de PR e Issues

Guía a los contribuidores para detallar qué problemas resuelven sus aportes.

Beneficios Fundamentales del Flujo Abierto



Reproducibilidad

Permite a terceros verificar resultados y ejecutar experimentos exactos.



Trazabilidad

Historial transparente sobre quién, cuándo y por qué modificó el proyecto.



Credibilidad

Aumenta la confianza en publicaciones mediante código auditable.



Democratización

Facilita la contribución global de cualquier estudiante o investigador.

¿Cuándo Integrar este Flujo?

Proyectos de Asignatura

Trabajos en equipo con reparto claro de tareas mediante ramas y Forks individuales.

Material Suplementario

Publicación de código complementario a artículos científicos e informes técnicos.

Desarrollo Open Source

Proyectos abiertos donde cualquier miembro de la comunidad puede aportar mejoras.

Repositorios Destacados de Referencia

NASA JPL Open Source Rover

Diseño abierto de rover mecatrónico a escala con contribuciones comunitarias globales.

github.com/nasa-jpl/open-source-rover

Vivero Smart – Tecnológico de Tijuana

Sistema de automatización agrícola colaborativo de código abierto.

github.com/tectijuana/viverosmarth-viveroitt

Referencias Bibliográficas

Blischak JD, Davenport ER, Wilson G (2019) *A Quick Introduction to Version Control with Git and GitHub*. PLoS Comput Biol 15(1): e1004668.

Ram, K. (2013) *Git can facilitate greater reproducibility and increased transparency in science*. Source Code Biol Med 8, 7.

Wilson G, et al. (2017) *Good enough practices in scientific computing*. PLoS Comput Biol 13(6): e1005510.

Chacon, S., & Straub, B. (2014) *Pro Git (2ª ed.)*. Apress. <https://git-scm.com/book/en/v2>



¡Gracias por su atención!

¿Preguntas o comentarios sobre flujos de trabajo en GitHub?

 sergioandres.nohpuch@gmail.com

 [CONTRIBUTING.md Listo](#)